

Memory Attention

Jiale Kang

kangjiale827@gmail.com

memory-attention

Abstract

Language models typically construct attention values from contextual hidden states, even when some of their content may be reusable across contexts. We investigate whether token-indexed memory can replace the dedicated value projection when complemented by contextual information. We propose **Memory Attention (MA)**, which forms values by combining layer-specific token memory with contextual keys. The memory supplies token-specific representations, while the keys preserve context dependence. At inference, normalization can be folded into the memory tables, reducing value construction to lookup and addition. Token-indexed retrieval also enables CPU offloading with prefetching, reducing GPU parameter storage. Under matched training token budgets and with additional memory parameters, experiments across attention configurations show improved language modeling and average downstream performance.

1 Introduction

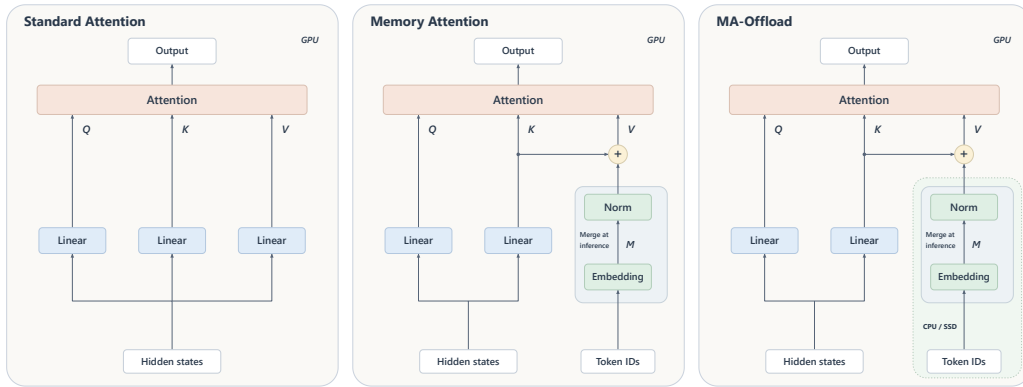


Figure 1: Standard Attention, Memory Attention, and MA-Offload. Memory Attention replaces the value projection with $V = K + \text{Norm}(M)$, where M is retrieved by token ID. At inference, normalization is merged into the embedding table, which MA-Offload stores on CPU or SSD. Additional components used in experiments, including RoPE, are omitted.

Improving language model quality under limited computation and GPU memory remains a central challenge. One approach is to expand model capacity through lookup-based representations, which provide additional learned parameters without proportional increases in dense computation. Recent methods, including Value Embedding [7], DeepEmbed [2], Per-Layer Embeddings (PLE) [6], STEM [11], and Engram [3], explore this direction through learned embeddings and memory modules. These developments raise a complementary question: can explicit memory replace part of an existing computation, rather than only supplement it?

Self-attention provides a natural setting for this question. Viewed as a content-addressable memory, attention uses query-key interactions to determine which information to retrieve and values to supply

the content to be aggregated. Standard attention constructs values from contextual hidden states through a dedicated linear projection [13]. We hypothesize that token-specific representations shared across contexts can supply part of this content, while an existing contextual representation supplies the remainder. In particular, keys already encode contextual information for attention. Can they also provide the contextual component of values, allowing token-indexed memory to replace the dedicated value projection?

We propose **Memory Attention (MA)**, which forms values by adding learned, layer-specific token memory to contextual keys. Each layer maintains its own embedding table, allowing the same token to have distinct stored representations at different depths. The retrieved embeddings contribute token-specific content, while the keys preserve context dependence. Unlike value embeddings that supplement projected values, MA removes the independent value projection and reuses the key representation. It introduces explicit memory directly into the value pathway while preserving the standard attention weighting and aggregation rules.

This representation changes both value construction and parameter placement. MA replaces a dense value projection with embedding lookup, normalization, and addition. At inference, normalization can be precomputed and folded into the memory tables, leaving only lookup and addition for online value construction. Although the tables increase total parameter storage, their entries are addressed solely by token IDs and layer indices. We exploit this property in **MA-Offload**, which stores the tables in CPU memory and prefetches the required entries before the corresponding layer uses them. This allows additional memory capacity to reside outside the GPU, with a latency cost determined by retrieval, transfer, and their overlap with computation. The additive representation also permits values to be reconstructed from retained keys and token memory. We analyze this possibility as **MA-Recall**, an extension that trades repeated retrieval and reconstruction for reduced persistent value-cache storage.

We evaluate MA through language modeling, downstream tasks, retrieval, and inference measurements. Across the evaluated attention configurations, MA improves both language modeling perplexity metrics and average downstream accuracy under matched training token budgets, while introducing additional memory parameters. Retrieval experiments also show gains within the training context window and at twice its length. Inference measurements demonstrate that CPU offloading can accommodate the larger MA parameterization with lower GPU parameter storage than the standard baseline, while introducing decoding overhead. These results characterize the quality and efficiency of the combined design; they do not isolate the contribution of its structure from the increase in parameter capacity.

Our main contributions are as follows:

- We propose **Memory Attention**, which combines layer-specific token memory with contextual keys to replace the dedicated value projection while preserving the standard attention weighting and aggregation rules.
- We develop and evaluate **MA-Offload**, which exploits token-indexed retrieval to place memory tables in CPU memory and prefetch the required entries. We further analyze **MA-Recall** as a potential extension for reconstructing values without a persistent value cache.
- We evaluate model quality and inference behavior, and analyze the trade-offs among parameter capacity, value-construction arithmetic, GPU parameter storage, memory traffic, and persistent cache storage.

2 Background

We review self-attention and three mechanisms for incorporating token-indexed memory. Layer indices are omitted throughout; embedding tables and transformations may differ across layers.

Self-Attention. Given hidden states $\mathbf{X}$, self-attention computes queries, keys, and values through linear projections:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V. \quad (1)$$

For a single head, the attention output is

$$\text{Attn}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{C}\right) \mathbf{V}, \quad (2)$$

where d_k is the key dimension and $\mathbf{C}$ is the causal mask. Query–key interactions determine the aggregation weights, while the values supply the content to be aggregated. All three representations are derived from the current hidden states.

Value Embeddings. We use $\mathbf{E}$ to denote a learnable embedding table and $\mathbf{E}[i]$ to denote an embedding lookup that retrieves its i -th row. Value embeddings supplement projected values with token-specific representations from $\mathbf{E} \in \mathbb{R}^{N \times d_v}$:

$$\mathbf{v}_{e,t} = \mathbf{E}[s_t], \quad \tilde{\mathbf{v}}_t = (1 - \lambda)\mathbf{v}_t + \lambda\mathbf{v}_{e,t}. \quad (3)$$

where s_t is the token ID and λ controls the embedding contribution. This combines contextual values with token-specific representations while retaining the original value projection.

Per-Layer Embeddings. Per-Layer Embeddings (PLE) introduce a learnable embedding table $\mathbf{E} \in \mathbb{R}^{N \times d_e}$ at each layer, where d_e is the embedding dimension. Omitting layer indices, normalization, and scaling, the update is

$$\mathbf{e}_t = \mathbf{E}[s_t] + \mathbf{P}\mathbf{x}_t^{(0)}, \quad (4)$$

$$\Delta\mathbf{h}_t = \mathbf{W}_o [\text{GELU}(\mathbf{W}_g \mathbf{h}_t) \odot \mathbf{e}_t], \quad (5)$$

where $\mathbf{x}_t^{(0)}$ is the input embedding and $\mathbf{h}_t$ is the hidden state at the injection point. The lookup and projected input embedding provide token-specific information, while the hidden-state gate modulates its contribution to the update.

Engram. Engram retrieves learned memory using local n -gram patterns. Let $\mathbf{E} \in \mathbb{R}^{H \times d_m}$ denote a learnable memory table with H entries of dimension d_m . Abstracting the multiple lookups in the full architecture, a single retrieval is

$$\mathbf{m}_t = \mathbf{E}[h(s_{t-n+1}, \dots, s_t)], \quad (6)$$

where h maps a local n -gram to a table index. The retrieved memory is integrated through contextual gating. Because lookup addresses depend only on token IDs, retrieval can be prefetched before the corresponding hidden states become available. This allows memory capacity to grow without a proportional increase in dense computation.

3 Memory Attention

Memory Attention (MA) replaces the dedicated value projection with a combination of contextual keys and layer-specific token memory. We first define this construction and its integration into attention, then analyze its computational and storage costs. We omit layer indices unless needed; each MA layer has its own memory table and learned transformations.

3.1 Memory-Based Value Construction

Let $\mathbf{X} \in \mathbb{R}^{T \times d}$ denote the hidden states of a sequence with token IDs $\mathbf{s} = (s_1, \dots, s_T)$. Let H_{KV} denote the number of key/value heads and d_h their head dimension. We assume matching key and value dimensions and write $d_v = H_{KV}d_h$ for the total dimension across key/value heads.

Each MA layer maintains a learnable memory table $\mathbf{E} \in \mathbb{R}^{N \times d_v}$, where N is the vocabulary size. The token memory is

$$\mathbf{M} = \text{Norm}(\mathbf{E}[\mathbf{s}]) \in \mathbb{R}^{T \times d_v}, \quad (7)$$

where $\mathbf{E}[\mathbf{s}]$ retrieves the rows indexed by the input tokens. Norm applies RMSNorm independently to each token vector within each key/value head. The memory tables and normalization parameters are learned jointly with the rest of the model. Within a layer, the same token retrieves the same memory vector across positions and contexts; separate tables allow different layers to learn distinct representations.

MA computes queries and keys from the hidden states, then constructs values by addition:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{K} + \mathbf{M}. \quad (8)$$

The memory supplies token-specific representations, while the keys provide dynamically computed information from the current hidden states. These hidden states incorporate contextual information and intermediate representations accumulated through preceding layers. The key contribution therefore reflects both the current context and the computation performed up to the current depth. Layer-specific memory complements this contribution with stored token representations. Unlike supplementing projected values with token embeddings, MA removes the independent value projection $\mathbf{W}_V$ and jointly learns the key projection and memory table to support value construction.

For attention with rotary positional embeddings (RoPE), positional transformations are applied to the queries and keys used for attention scoring:

$$\mathbf{Q}^R = \text{RoPE}(\mathbf{Q}), \quad \mathbf{K}^R = \text{RoPE}(\mathbf{K}), \quad \mathbf{V} = \mathbf{K} + \mathbf{M}. \quad (9)$$

Values are constructed from $\mathbf{K}$ before this transformation, while $\mathbf{Q}^R$ and $\mathbf{K}^R$ are used to compute attention weights. The resulting values are passed to the original attention operator, leaving its weighting and aggregation rules unchanged.

3.2 Connection to MLA through Attention Aggregation

The connection between MA and MLA can be understood through their use of a shared representation for attention scoring and content aggregation. For a single head, omitting positional components and attention masks, MLA constructs keys and values from a compressed latent representation $\mathbf{C}$:

$$\mathbf{K} = \mathbf{C}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{C}\mathbf{W}_V. \quad (10)$$

By associativity, its attention computation can be written as

$$\mathbf{A}_{\text{MLA}} = \text{softmax}\left(\frac{(\mathbf{Q}\mathbf{W}_K^\top)\mathbf{C}^\top}{\sqrt{d_k}}\right), \quad (11)$$

$$\mathbf{O}_{\text{MLA}} = (\mathbf{A}_{\text{MLA}}\mathbf{C})\mathbf{W}_V. \quad (12)$$

The key mapping can therefore be absorbed into the query-side computation, while the value mapping can be applied after aggregation. In this form, attention retrieves information directly from the shared latent representation, and the value mapping transforms the retrieved content.

To make the comparison explicit, let $\mathbf{Z}$ denote the shared representation being aggregated: $\mathbf{Z}_{\text{MLA}} = \mathbf{C}$ for MLA and $\mathbf{Z}_{\text{MA}} = \mathbf{K}$ for MA, where the latter denotes keys before positional transformation. The two outputs then take the forms

$$\mathbf{O}_{\text{MLA}} = (\mathbf{A}_{\text{MLA}}\mathbf{Z}_{\text{MLA}})\mathbf{W}_V, \quad (13)$$

$$\mathbf{O}_{\text{MA}} = \mathbf{A}_{\text{MA}}(\mathbf{Z}_{\text{MA}} + \mathbf{M}) \quad (14)$$

$$= \mathbf{A}_{\text{MA}}\mathbf{Z}_{\text{MA}} + \mathbf{A}_{\text{MA}}\mathbf{M}. \quad (15)$$

Both formulations use a shared contextual representation to support attention scoring and content retrieval. Their difference lies in how the retrieved content becomes the output: MLA applies a learned linear value mapping, whereas MA directly aggregates keys and supplements the result with token memory under the same attention weights. The shared notation describes their functional roles; it does not imply identical representations, dimensions, or attention weights.

This comparison offers a rationale for MA. As discussed above, the keys contain contextual information and intermediate representations accumulated through preceding layers, providing a dynamically computed source of output content. Token memory supplies an additional, layer-specific source of content without a dedicated value projection. Although each memory entry is fixed across contexts, its aggregated contribution depends on the attention weights. The key projection and memory table are learned jointly, allowing them to adapt to these complementary roles. MA thus explores an alternative to a separate linear value mapping through direct key reuse and an additive memory read, rather than an algebraically equivalent reformulation of MLA.

Metric	Standard	MA
Value construction	$\mathbf{XW}_V$	$\mathbf{K} + \overline{\mathbf{E}}[\mathbf{s}]$
Trainable parameters	Ldd_v	$L(Nd_v + p_{\text{norm}})$
Stored inference parameters	Ldd_v	LNd_v
Online value-construction FLOPs	$\approx 2LSdd_v$	LSd_v

Table 1: Value-construction costs across L replaced layers. Shared components, including the key projection, are excluded. Each multiplication or addition counts as one FLOP. Lookup overhead and memory traffic are excluded. Inference costs apply to newly processed tokens and do not include reconstruction of historical values.

3.3 Computation and Parameter Storage

We compare the value-construction costs of MA and standard attention across L replaced layers. Let S denote the number of newly processed tokens: $S = BT$ for prefill with batch size B and sequence length T , and $S = B$ for one decoding step. We assume bias-free projections and denote the number of learned normalization parameters per layer by p_{norm} . Shared components, including the key projection, are excluded from this comparison.

Parameter capacity. Standard attention uses Ldd_v parameters for the value projections. MA replaces them with $L(Nd_v + p_{\text{norm}})$ memory and normalization parameters. When $N > d$, this increases parameter capacity and total parameter storage. The additional capacity is accessed through token lookup rather than a dense projection.

Training computation. The standard value projection requires approximately $6LSdd_v$ FLOPs for the forward pass and the input and weight gradients. MA instead requires $\mathcal{O}(LSd_v)$ arithmetic for head-wise normalization, addition, and gradient accumulation over accessed memory entries. This comparison excludes optimizer updates, gradient-buffer initialization, and memory traffic. In particular, lookup-based computation does not by itself imply sparse gradient storage or sparse optimizer states.

Inference computation. At inference, both the memory tables and normalization parameters are fixed. Because normalization acts independently on each retrieved row, it can be folded into the tables:

$$\overline{\mathbf{E}}[i] = \text{Norm}(\mathbf{E}[i]), \quad i = 1, \dots, N. \quad (16)$$

Online value construction then becomes

$$\mathbf{V} = \mathbf{K} + \overline{\mathbf{E}}[\mathbf{s}], \quad (17)$$

requiring only lookup and element-wise addition. Table 1 summarizes the parameter and inference arithmetic costs.

The reduction in online value-construction arithmetic is approximately

$$\Delta F_V \approx LSd_v(2d - 1). \quad (18)$$

If F_{common} denotes the FLOPs of all unchanged components for the same workload, the complete-model FLOP reduction is

$$\eta_{\text{model}} \approx \frac{LSd_v(2d - 1)}{F_{\text{common}} + 2LSdd_v}. \quad (19)$$

This arithmetic reduction does not directly determine latency, which also depends on memory access, kernel execution, and synchronization.

3.4 MA-Offload

The folded memory tables need not remain on the GPU. Their lookup addresses depend only on token IDs and layer indices, so the required entries can be identified before the corresponding hidden states are available. **MA-Offload** stores the folded tables in CPU memory and prefetches the required vectors to the GPU, allowing retrieval and transfer to overlap with model computation.

Let b_w denote the number of bytes per stored memory element. Offloading moves

$$P_{\text{offloaded}} = b_w L N d_v \quad (20)$$

bytes of memory-table parameters from GPU to CPU storage. It does not change the total parameter count or the value-construction arithmetic. The GPU still requires temporary lookup and transfer buffers.

Without caching or deduplication, the logical host-to-device payload for S newly processed tokens is

$$D_{\text{Offload}} = b_w L S d_v. \quad (21)$$

With a conventional KV cache, historical values remain cached, so each decoding step retrieves memory only for the new tokens. The resulting payload is $b_w L B d_v$ bytes per step.

MA-Offload reduces GPU parameter residency while retaining the historical key and value cache. Its latency cost depends on CPU lookup performance, transfer bandwidth, synchronization, and the overlap available in the execution schedule. Consequently, moving the tables off the GPU reduces parameter storage but does not guarantee a reduction in either total GPU memory usage or inference latency.

3.5 MA-Recall: An Extension for Value Reconstruction

The additive value representation also permits reconstruction of historical values. We analyze this possibility as **MA-Recall**, an extension that avoids a persistent value cache by retaining sufficient key information and the historical token IDs. MA-Recall is not included in the inference measurements reported in this work. Given historical content keys and token IDs, values are recovered as

$$\mathbf{V}1 : T = \mathbf{K}1 : T + \overline{\mathbf{E}}[s_{1:T}]. \quad (22)$$

A practical implementation must also support the keys required for attention scoring. With standard RoPE, a cached rotated key can in principle be mapped back to its content representation using the inverse positional rotation. Alternatively, content keys can be retained and rotated when needed. Either choice introduces positional-transformation work that must be included in implementation-level cost measurements. Retaining both representations would require additional cache storage. Let b_c denote the number of bytes per cached key or value element. For equal key and value dimensions and precision, a conventional content KV cache occupies $2b_c L B T d_v$ bytes. Retaining one key representation per historical position reduces this component to $b_c L B T d_v$ bytes, a reduction of 50%. This accounting excludes token IDs, position metadata, temporary reconstruction buffers, and any additional positional-key storage. Blockwise reconstruction could avoid materializing the complete value history on the GPU. The storage reduction comes at the cost of repeated memory access and reconstruction. Let R denote the number of historical positions whose values are reconstructed per sequence in a decoding step, assuming the same count across layers. Reconstruction requires approximately $L B R d_v$ additions, excluding any positional transformations. For full causal attention, $R = T$; for restricted attention patterns, R is the number of positions actually reconstructed. When memory tables are also CPU-offloaded, the corresponding logical transfer payload is

$$D_{\text{Recall}} = b_w L B R d_v, \quad (23)$$

without caching or deduplication. MA-Recall therefore trades persistent cache storage for additional retrieval, transfer, and reconstruction work. Its practical benefit depends on the attention pattern, cache representation, memory placement, and reconstruction implementation.

4 Experiments

We evaluate MA in terms of language modeling and downstream performance, training efficiency, retrieval, and inference behavior. The quality evaluations examine whether token memory and shared keys can effectively replace the dedicated value projection. The efficiency evaluations characterize the computational and parameter-storage trade-offs of GPU-resident and CPU-offloaded memory tables.

Backbone	Variant	Params.	Lamb. PPL ↓	WikiText PPL ↓	Lamb. Acc. ↑	ARC-E Acc. ↑	ARC-C Acc. _n ↑	HellaSwag Acc. _n ↑	PIQA Acc. ↑	WinoGrande Acc. ↑	OBQA Acc. ↑	Avg. ↑
<i>24 layers, hidden size 1,024; 10B training tokens; 0.5M tokens per batch</i>												
MHA	Standard	373M	44.35	31.55	31.59	53.91	26.96	36.86	65.18	49.09	21.20	40.68
	Memory	1,135M	42.78	28.64	32.25	56.44	25.85	38.41	66.00	50.36	20.40	41.39
GQA	Standard	349M	50.46	31.71	30.86	54.21	27.47	36.59	65.83	48.93	22.40	40.90
	Memory	729M	43.68	29.45	30.97	56.27	27.65	38.07	65.18	51.62	20.80	41.51
MQA	Standard	336M	50.38	31.84	31.32	54.17	26.88	35.98	64.53	53.28	21.60	41.11
	Memory	526M	45.93	29.82	31.55	55.05	26.62	37.55	65.56	52.25	20.80	41.34
Gate	Standard	398M	41.88	29.67	32.10	56.06	26.71	38.21	65.23	50.91	20.60	41.40
	Memory	1160M	36.60	27.48	33.73	56.69	27.99	39.73	66.65	51.78	23.60	42.88
<i>24 layers, hidden size 2,048; 20B training tokens; 1M tokens per batch</i>												
MHA	Standard	1,364M	18.59	21.52	40.50	66.67	31.31	46.98	69.91	53.67	34.40	49.06
	Memory	2,836M	18.15	20.79	41.59	66.04	33.96	48.38	69.42	54.78	37.40	50.22

Table 2: Zero-shot language modeling and downstream results at matched training token budgets. Parameter counts include memory tables and differ within pairs. Accuracies are percentages; Acc_n denotes normalized accuracy; Avg. averages the seven accuracy scores. Bold marks the better result in each Standard–Memory pair. Experiments on additional backbones, including swa and linear attention, as well as larger models and mixture-of-experts (MoE) architectures, are ongoing.

4.1 Experimental Setup

Training configurations. We train models on NVIDIA H800 GPUs using the flash-linear-attention [14] framework. The backbone consists of attention blocks and gated MLPs, with rotary positional embeddings (RoPE) and RMSNorm. Detailed pretraining settings are provided in Appendix A, and model configurations are summarized in Table 2. Within each configuration, Standard and MA use matched training token budgets, while MA introduces additional memory parameters.

Evaluation benchmarks. We evaluate language modeling and downstream performance in a zero-shot setting using lm-evaluation-harness [5]. We report perplexity on LAMBADA [10] and WikiText [8], together with LAMBADA word-prediction accuracy. Downstream benchmarks include ARC-Easy and ARC-Challenge [4], HellaSwag [15], PIQA [1], WinoGrande [12], and OpenBookQA [9]. We use normalized accuracy for ARC-Challenge and HellaSwag and accuracy for the remaining tasks. The average score is the unweighted mean of the seven accuracy-based scores, excluding perplexity.

4.2 Language Modeling

Table 2 shows that MA improves both perplexity metrics and the average accuracy-based score across all three configurations. For example, WikiText perplexity decreases from 31.55 to 28.64 for the smaller MHA model, while LAMBADA perplexity decreases from 50.46 to 43.68 for GQA. The larger model also improves on both language modeling metrics.

The average accuracy gains are 0.71 percentage points for the smaller MHA model, 0.61 for GQA, and 1.16 for the larger MHA model. Improvements vary across tasks: at the larger scale, MA gains 3.00 points on OpenBookQA and 2.65 points on ARC-Challenge, while slightly reducing ARC-Easy and PIQA accuracy. Thus, the aggregate improvements do not imply gains on every benchmark.

These results support the effectiveness of the combined MA design under the reported training budgets. Because MA includes additional memory parameters, they do not isolate the contribution of its structure from the increase in parameter capacity.

These ratios describe the number of training tokens needed to reach the selected loss levels. They do not directly measure wall-clock training speed. MA also reduces value-construction arithmetic, as analyzed in Section 3.3, but realized training time additionally depends on memory access, gradient handling, and optimizer updates for the larger parameter set.

Retrieval and Context-Length Extrapolation We evaluate retrieval within the training context window and under context-length extrapolation using three single-needle NIAH tasks. The evaluated

models have 24 layers and a hidden dimension of 1,024 and are trained on 10B tokens with a context length of 2,048. Evaluation lengths are 1,024, 2,048, and 4,096 tokens. The first two fall within the training window, while the last tests extrapolation to twice the training length.

As shown in Table 3, MA outperforms Standard across all nine task-length combinations. Within the training window, its mean scores are 97.4% at 1K and 96.8% at 2K, compared with 82.6% and 82.1% for Standard. The largest individual gain occurs on `niah_single_3` at 1K, where performance increases from 62.4% to 98.2%.

At 4K, MA retains an advantage on all three tasks, with a mean score of 41.9% compared with 25.9% for Standard. Both models nevertheless degrade substantially beyond the training window. These results demonstrate improved retrieval on the evaluated tasks, including under $2\times$ length extrapolation, but do not establish robust retrieval at longer context lengths.

Variant	niah_single_1			niah_single_2			niah_single_3		
	1K	2K	4K [†]	1K	2K	4K [†]	1K	2K	4K [†]
Standard	0.998	0.844	0.414	0.856	0.914	0.232	0.624	0.704	0.130
MA	1.000	0.994	0.510	0.940	1.000	0.454	0.982	0.910	0.294

Table 3: Zero-shot single-needle NIAH scores for models trained with a 2K context window. 1K, 2K, and 4K denote 1,024, 2,048, and 4,096 tokens, respectively. [†] marks extrapolation beyond the training context length. Higher scores are better; bold indicates the better result in each column.

4.3 Training and Inference Efficiency



Figure 2: Training and inference comparison of MA and Standard. At the matched-loss operating points shown, MA achieves $1.42\times$ and $1.16\times$ token efficiency for L24-D1024 and L24-D2048, respectively. The remaining panels report training FLOPs, parameter placement, and model forward-pass latency for L24-D2048. MA-Offload stores memory-table parameters in CPU memory.

We examine whether MA can expand parameter capacity while reducing value-construction arithmetic and GPU parameter storage. We first compare training token efficiency, then evaluate inference latency and parameter placement. The inference results use a prototype implementation with prefetching and computation-transfer overlap.

Training efficiency. We compare the number of training tokens required to reach a matched loss. For a target loss ℓ , let $n_{\text{Standard}}(\ell)$ and $n_{\text{MA}}(\ell)$ denote the corresponding token counts. We define token efficiency as

$$\rho_{\text{token}}(\ell) = \frac{n_{\text{Standard}}(\ell)}{n_{\text{MA}}(\ell)}. \quad (24)$$

At the operating points shown in Figure 2, MA achieves token efficiencies of $1.42\times$ and $1.16\times$ for L24-D1024 and L24-D2048, respectively. These correspond to approximately 29.6% and 13.8% fewer training tokens to reach the selected loss levels. The comparisons use models with additional memory parameters and measure token efficiency rather than wall-clock training speed.

Inference configurations. We compare standard attention (STANDARD), MA with memory tables stored on the GPU (MA), and MA with CPU-offloaded memory tables (MA-OFFLOAD). Both MA variants fold head-wise RMSNorm into the tables before inference, reducing online value construction to lookup and addition. MA-Offload additionally uses a prefetch pipeline to overlap CPU retrieval and host-to-device transfer with model computation. All three configurations retain a conventional KV cache; these measurements do not include MA-Recall.

Configuration	Latency (ms)		Parameters (M)			Parameter storage (MiB)	
	Prefill	Decode	GPU	CPU	Total	GPU	CPU
STANDARD	90.970	17.636	1,364.298	0.000	1,364.298	2,602.38	0.00
MA	88.202	17.788	2,836.498	0.000	2,836.498	5,410.38	0.00
MA-OFFLOAD	90.686	17.189	1,263.634	1,572.864	2,836.498	2,410.38	3,000.00

Table 4: Model forward-pass latency and parameter placement for the reference workload: batch size 8, prefill length 2,048, and one decoding step with a cached history of 2,048 tokens. Timing and parameter counts cover the complete model, including the input embedding, final normalization, and output head. Parameter storage is reported at BF16 precision and excludes KV caches, activations, and transfer buffers. M denotes 10^6 parameters; MiB denotes 2^{20} bytes.

Experiments use a single NVIDIA H800 GPU with BF16 precision, PyTorch 2.9.1 (CUDA 12.6), and FlashAttention 2.8.3. The model has 24 layers, a hidden size of 2,048, 32 query heads, and 32 key/value heads. Each layer uses pre-normalized residual blocks and a gated MLP with an intermediate size of 5,632. The vocabulary contains 32,000 tokens, and the input embedding and output head are untied.

For the reference workload, we use batch size 8, prefill length 2,048, and a cached decoding history of 2,048 tokens. Each decoding measurement processes one new token per sequence. The offload group size is one layer for prefill and four layers for decoding, with a prefetch depth of four in both stages. MA performs layer-wise memory lookup.

Measurement scope. We measure the model forward pass, including the input embedding, Transformer layers, final RMSNorm, and output head. Prefill computes logits at every input position. For MA-Offload, the measurement includes memory retrieval and transfer during the forward pass. Timing excludes table initialization, normalization folding, decode-prefix construction, sampling, and token-ID transfers from GPU to CPU. Token IDs are available on both devices before measurement. The reported latencies therefore describe model forward passes rather than end-to-end serving.

Parameter-storage estimates include all parameters retained for inference, including folded memory tables, and distinguish their storage devices. They exclude KV caches, activations, and transfer buffers and therefore do not measure total or peak GPU memory usage.

Value-construction overhead. MA replaces the dedicated value-projection matrix multiplication with memory lookup and element-wise addition. As shown in Table 4, MA reduces measured prefill latency from 90.970 to 88.202 ms (3.04%), while decode latency increases slightly from 17.636 to 17.788 ms (0.86%). Thus, despite increasing total parameter count to approximately $2.08\times$ that of Standard, MA maintains comparable forward-pass latency at this operating point. The results also illustrate that reduced value-construction arithmetic does not uniformly translate into lower latency, as memory access and execution overhead remain relevant.

GPU parameter storage. MA-Offload moves 1,572.864M memory-table parameters to CPU memory, corresponding to 3,000 MiB at BF16 precision. This reduces GPU parameter storage from 5,410.38 MiB for MA to 2,410.38 MiB, a reduction of 55.45%. Relative to Standard, GPU parameter storage decreases by 192 MiB (7.38%), reflecting the removal of the dedicated value projections. MA-Offload therefore accommodates approximately $2.08\times$ the total parameters of Standard with less GPU parameter storage, while using additional CPU memory.

Transfer overhead and overlap. Offloading introduces CPU memory access and host-to-device transfers. Since lookup addresses depend only on token IDs and layer indices, our prototype prefetches memory entries and overlaps retrieval and transfer with model computation to limit the overhead exposed on the critical path.

At the reference workload, MA-Offload records prefill and decode latencies of 90.686 and 17.189 ms, respectively, compared with 90.970 and 17.636 ms for Standard. These correspond to measured latency reductions of 0.31% and 2.53%. The results demonstrate that, in this configuration, the additional memory traffic can be accommodated without a net increase in measured forward-pass

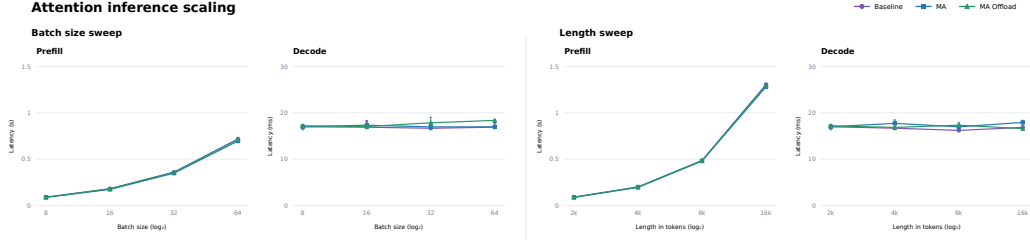


Figure 3: Model forward-pass latency of Standard, MA, and MA-Offload. Batch size varies at length 2K (left), and length varies at batch size 8 (right). Each pair of panels reports prefill and decoding latency. Points indicate medians; error bars show the minimum and maximum across measurement rounds. Horizontal axes use a logarithmic scale with base 2; 1K denotes 1,024 tokens. Lower is better.

latency. These aggregate timings combine the effects of reduced value-projection computation, memory access, transfer, and scheduling; they do not establish that communication is cost-free or isolate the contribution of overlap.

Implementation scope. Our implementation is a prototype for examining the storage–latency trade-off. The measurements demonstrate the feasibility of expanding parameter capacity and reducing GPU parameter storage while maintaining near-baseline inference latency. Further optimization of lookup, buffering, and transfer scheduling may improve performance, but is not evaluated here. The small timing differences should be interpreted as measurements at the reported operating point rather than consistent speed advantages across workloads.

References

- [1] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *AAAI*, 2020.
- [2] BoPeng. DeepEmbed. X post, 2025.
- [3] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026.
- [4] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [5] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024.
- [6] Gemma Team. Gemma 3n. Google DeepMind model documentation, 2025.
- [7] KoszarskyB. Layerwise token value embeddings. X post, 2024. Announcement accompanying the modded-nanogpt implementation.
- [8] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.
- [9] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*, 2018.
- [10] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset: Word prediction requiring a broad discourse context. In *ACL*, 2016.
- [11] Ranajoy Sadhukhan, Sheng Cao, Harry Dong, Changsheng Zhao, Attiano Purpura-Pontoniére, Yuandong Tian, Zechun Liu, and Beidi Chen. STEM: Scaling transformers with embedding modules, 2026.
- [12] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.
- [14] Songlin Yang and Yu Zhang. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, January 2024.
- [15] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *ACL*, 2019.

A Pretraining Settings

Data and optimization. We conduct pretraining experiments on FineWeb-10BT. Unless otherwise specified, models are trained for 20,480 optimization steps with a sequence length of 2,048 and a global batch size of 256 sequences. We use AdamW with a peak learning rate of 3×10^{-4} and $\epsilon = 10^{-15}$. The learning rate is warmed up over the first 1,024 steps and then decayed using a cosine schedule to 10% of its peak value. The global gradient norm is clipped at 1.0. Within each backbone configuration, Standard and MA use the same training settings and random seed for controlled comparison. Table 5 summarizes the default training hyperparameters.

Hyperparameter	Value
Dataset	FineWeb-10BT
Optimizer	AdamW
Peak learning rate	3×10^{-4}
Adam ϵ	10^{-15}
Learning-rate schedule	Cosine decay
Warmup steps	1,024
Minimum learning-rate ratio	0.1
Global batch size (sequences)	256
Sequence length	2,048
Training steps	20,480
Gradient clipping threshold	1.0
Random seed	42

Table 5: Default pretraining hyperparameters. The minimum learning-rate ratio is relative to the peak learning rate.

Backbone and attention configurations. For the hidden-dimension-1,024 backbone, all models use 24 Transformer layers. MHA uses 16 query heads and 16 key/value heads, while GQA uses 16 query heads and eight key/value heads; both use a head dimension of 64. MQA uses four query heads sharing a single key/value head, with a head dimension of 256. Each key/value head is therefore associated with one, two, and four query heads in MHA, GQA, and MQA, respectively. Keys and values have matching per-head dimensions within each configuration. The models use RoPE with a base of 10,000, RMSNorm with $\epsilon = 10^{-6}$, and untied input embeddings and output heads. Table 6 summarizes these configurations. Standard and MA use identical head settings within each comparison; comparisons across attention configurations also reflect differences in head count and dimension.

Configuration	MHA	GQA	MQA
Number of layers	24	24	24
Hidden dimension	1,024	1,024	1,024
Query heads (H_Q)	16	16	4
KV heads (H_{KV})	16	8	1
KV head dimension (d_h)	64	64	256
Query heads per KV head	1	2	4
Total KV dimension (d_v)	1,024	512	256
RoPE base	10,000	10,000	10,000
RMSNorm ϵ	10^{-6}	10^{-6}	10^{-6}
Tied input/output embeddings	No	No	No

Table 6: Attention configurations for the 24-layer backbone with hidden dimension 1,024. KV head dimension denotes the dimension of each key/value head, while $d_v = H_{KV}d_h$ denotes the total dimension of keys or values across heads. Standard and MA use the same head configuration within each comparison.

Memory configuration. Each MA layer maintains a token memory table $\mathbf{E} \in \mathbb{R}^{N \times d_v}$, where N is the vocabulary size and $d_v = H_{KV}d_h$. The memory-table widths are therefore 1,024, 512, and 256 for MHA, GQA, and MQA, respectively. Retrieved memory vectors are normalized independently within each key/value head and added to the corresponding keys before RoPE to construct values. Query heads sharing a key/value head also share its memory contribution. Memory tables are independent across layers and are learned jointly with the rest of the model.